VulnScan Lite

On-demand passive web vulnerability scanner

Project	VulnScan Lite - On-Demand Web Vulnerability Scanner
Programme	Persevex Internship - Cybersecurity domain
Author	Ayan Kumar Sarkar, BCA (Cyber Security), Sharda University
GitHub repository	https://github.com/sarkarayan1112-lab/VulnScan-Lite
Live application	https://vulnscan-lite-ypt7.onrender.com
Documented on	27 October 2026

Note: the live application runs on a free hosting plan, so the first visit after a period of inactivity can take 30 to 50 seconds to wake up.

1. Project overview

Many websites go live with basic security gaps such as missing security headers, an expiring certificate, or a server that announces its exact software version. Owners often find out only after an attack. VulnScan Lite lets anyone enter a web address and receive a plain-language health report with a grade from A to F, a list of passed and failed checks, and a how-to-fix tip for every failure.

The scanner is passive. It makes ordinary HTTPS requests and one TLS handshake, reads what the server already reveals, and never sends attack payloads. That keeps it safe and legal to use on a website you own.

2. Key features

Feature	What it does
Three scan modules	Security headers, SSL/TLS inspection, and CMS and server information, 10 checks in total.
Asynchronous scans	Each scan runs in a background thread. The page polls a status endpoint every 2 seconds, so the web server never freezes.
Report card	Animated gauge with a score out of 100 and a grade, plus tables of passed and failed checks.
Remediation tips	Every failed check shows a specific How to fix instruction.
PDF export	One click downloads a formatted report of the scan.
Abuse protection	Private and local addresses are blocked, and scan requests are rate limited.
Disclaimer banner	A banner on every page states: Only scan websites you own. This tool performs passive analysis only.

3. Scanning logic

Each check is a pass or a fail. Inside a module a pass adds 10 points and a fail subtracts 10, and that raw module score is kept in the scan data. The grade shown to the user comes from the overall pass rate, explained in section 3.4.

3.1 Security headers (module: scanner/headers.py)

The scanner sends one GET request to the target and looks at the response headers.

Header	Why it matters	Fix shown when missing
Content-Security-Policy	Tells the browser which sources of scripts and content are allowed. Limits cross-site scripting and injection.	Add a Content-Security-Policy header.
X-Frame-Options	Stops other sites from embedding the page in a frame, which prevents clickjacking.	Add X-Frame-Options: DENY.
Strict-Transport-Security	Forces browsers to use HTTPS for future visits.	Add Strict-Transport-Security: max-age=31536000; includeSubDomains.

3.2 SSL/TLS inspection (module: scanner/ssl_check.py)

The scanner opens a TLS connection on port 443 using Python's ssl library with certificate verification switched on, then inspects the result.

Check	Passes when	Fix shown when it fails
SSL certificate valid	The handshake succeeds and the certificate is trusted, not expired, and matches the host name.	Install a valid certificate from a trusted authority such as Let's Encrypt. The exact error is included.
Certificate expiry	More than 30 days remain before the expiry date.	Renew soon and use automatic renewal such as certbot.
TLS version	The connection negotiates TLS 1.2 or TLS 1.3.	Disable old TLS versions and enable only 1.2 or 1.3.
Cipher strength	The negotiated cipher uses at least 128 bits.	Configure the server to use strong ciphers (128-bit or higher).

If port 443 cannot be reached at all, the report shows a single failed HTTPS Available check instead of the four checks above.

3.3 CMS and server information (module: scanner/cms.py)

The scanner downloads the home page and examines the HTML and headers for technology fingerprints.

Check	How it works	Fix shown when it fails
CMS detection and version	Looks at the generator meta tag, the wp-content path, and the X-Generator header to identify WordPress, Drupal or Joomla. If a version is exposed it is compared with a minimum version (WordPress 6.6, Drupal 10.0, Joomla 5.0).	Update the CMS to the latest release. A site with no recognised CMS, or a hidden version, passes.
X-Powered-By header	Fails if the header exists. If it reveals a PHP version below 8.2, the report names it as outdated PHP.	Remove the header (expose_php = Off in PHP, proxy_hide_header in Nginx) and upgrade PHP.
Server header	Fails if the Server header contains a version number such as nginx/1.18.0.	Hide the version: server_tokens off in Nginx, ServerTokens Prod in Apache.

The minimum versions are fixed values inside the code and should be reviewed from time to time as new releases appear.

3.4 Scoring and grades

Overall score = (passed checks / total checks) x 100, rounded to the nearest whole number. The total is 10 when every module connects (3 headers, 4 SSL/TLS, 3 CMS and server).

Grade	A	B	C	D	F
Score	90 - 100	80 - 89	65 - 79	50 - 64	below 50

Example: a site that passes 7 of 10 checks scores 70 and receives grade C.

4. Architecture and request flow

- **1. Submit.** The browser sends the address to POST /api/scan.
- **2. Validate.** The server resolves the host name and rejects private, local, link-local, reserved and multicast addresses.
- **3. Queue.** A job ID is created, stored in memory with status queued, and a background thread starts the scan. The API answers immediately with the job ID.
- **4. Scan.** The thread runs the headers, SSL/TLS and CMS modules one after another. A module that crashes is reported as a failed check instead of stopping the scan.
- **5. Poll.** The page calls GET /api/scan/<id>/status every 2 seconds until the status is completed.
- **6. Report.** The page draws the gauge and tables. The PDF button calls GET /api/scan/<id>/pdf.

Endpoint	Method	Purpose
/	GET	Serves the web interface.
/api/scan	POST	Starts a scan from a JSON body such as {"url": "example.com"}. Limited to 5 requests per minute per visitor.
/api/scan/<id>/status	GET	Returns queued, running or completed, and the full result once completed.
/api/scan/<id>/pdf	GET	Downloads the PDF report for a completed scan.

5. Safety and responsible use

- **Passive analysis only.** No exploit payloads, brute force or crawling beyond the home page.
- **Server-side request forgery (SSRF) protection.** Because the server fetches whatever address a user types, localhost and internal network addresses are blocked before any request is made.
- **Rate limiting.** At most 5 scans per minute per visitor, with a clear message when the limit is reached. The app reads the real visitor address through the hosting proxy.
- **Request timeouts.** Every outgoing request and TLS connection times out after 10 seconds.
- **Safe output handling.** Text from scanned sites is placed into the page and the PDF as plain text, not as markup.
- **Visible disclaimer.** The banner reminds users to scan only websites they own.

6. Technology stack and repository layout

Area	Technology
Language	Python 3
Web framework	Flask, served in production by gunicorn
Scanning	requests, BeautifulSoup4, and Python's socket and ssl modules
Rate limiting	Flask-Limiter
PDF generation	ReportLab
Frontend	HTML, CSS and vanilla JavaScript with an SVG gauge
Hosting	Render (free web service), deployed from the GitHub main branch

Path	Contents
scanner/headers.py	Security header checks
scanner/ssl_check.py	SSL/TLS checks
scanner/cms.py	CMS and server information checks
app.py	API routes, SSRF protection, background jobs, scoring
report.py	PDF report builder
templates/index.html	Web interface
requirements.txt	Python dependencies

7. Screenshots

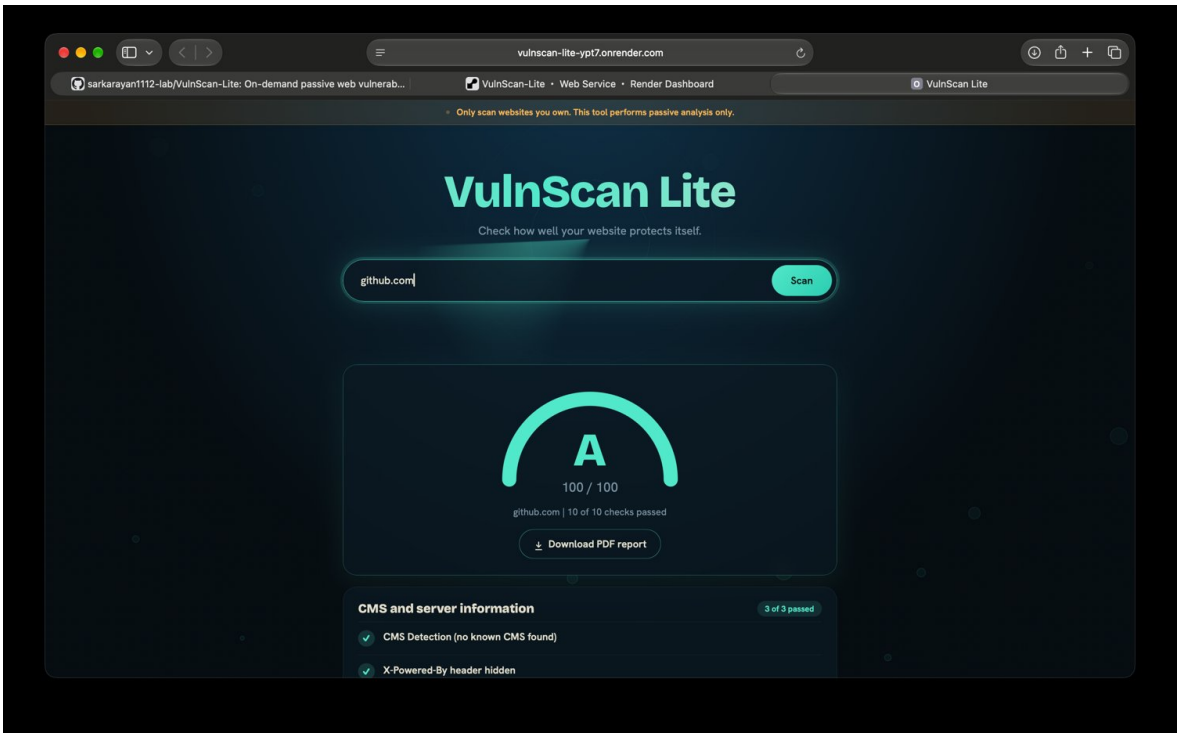


Figure 1. Live application: scan of github.com, grade A with a score of 100 out of 100 and the PDF download button.

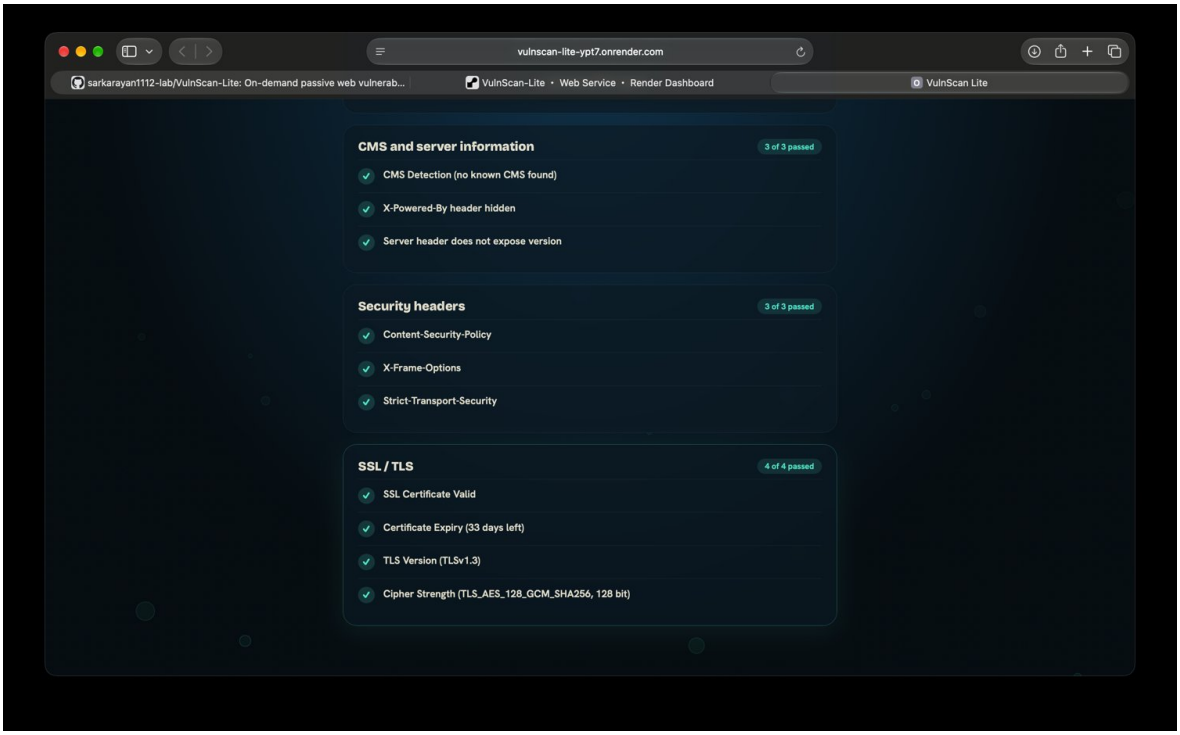


Figure 2. Detailed results for github.com: CMS and server information, security headers, and SSL/TLS.

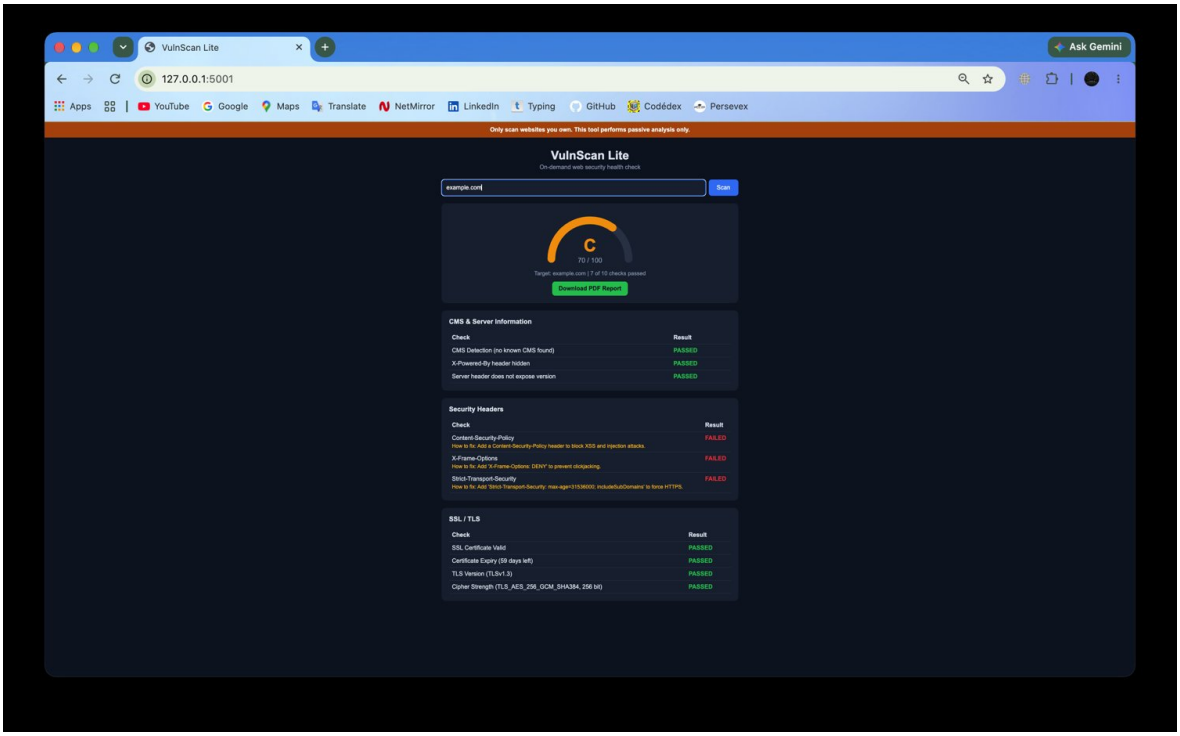


Figure 3. Scan of example.com during local testing (earlier interface version): grade C, three missing headers, each with a How to fix tip.

8. Current limitations and future work

- **Jobs live in memory.** Scan results are lost when the server restarts, and the app runs a single worker so every request can see the same jobs. Moving the queue to Redis with Celery or RQ would remove this limit and allow several workers.
- **No scan history yet.** Saving past scores per user, so improvement can be tracked over time, needs a database and logins.
- **Plain JavaScript frontend.** The interface is built without a framework to keep it light. A React version is a possible next step.
- **Version rules are static.** CMS and PHP minimum versions are set in code and need periodic updates.
- **Scope.** Only the home page is analysed, and only the checks in section 3 are performed.

Source code: https://github.com/sarkarayan1112-lab/VulnScan-Lite

Live application: https://vulnscan-lite-ypt7.onrender.com